지금까지 기획한 1번부터 7번까지의 완벽한 시나리오를 바탕으로, 허락된 도구(print, input, f-string, 리스트, if/elif/else, for/range)만을 사용하여 작성한 파이썬 코드입니다.

아직 배우지 않은 while문 대신 for i in range(100):과 break를 활용해 키오스크의 무한 반복을 훌륭하게 구현했습니다. 코드를 천천히 읽어보시면 우리가 나눈 이야기들이 어떻게 파이썬 언어로 번역되었는지 한눈에 보이실 거예요!

💻 파이썬 PC방 무인 정산기 전체 코드

Python

```python
# --- [초기 변수 및 리스트 설정] ---
# 좌석 10개의 상태를 리스트로 만듭니다.
seats = ["비어있음", "비어있음", "비어있음", "비어있음", "비어있음",
         "비어있음", "비어있음", "비어있음", "비어있음", "비어있음"]

# 먹거리 메뉴와 가격 리스트
food_menu = ["라면", "핫도그", "콜라", "소떡소떡"]
food_price = [3000, 2000, 1500, 2500]

cart = []           # 장바구니 리스트
my_seat = -1        # 내 좌석 번호 (-1은 아직 선택 안 함을 의미)
total_time = 0.0    # 남은 시간
total_payment = 0   # 총 결제할 금액
daily_revenue = 0   # 관리자용: 오늘 총매출
admin_pw = "0000"   # 관리자 비밀번호

print("=== 파이썬 PC방 무인 시스템을 시작합니다 ===")

# --- [메인 키오스크 반복문] ---
# while 대신 for와 range를 사용하여 넉넉하게 100번 반복하게 합니다.
for i in range(100):
    print("\n" + "="*30)
    print("[ 메인 메뉴 ]")
    print("1. PC 시간 충전")
    print("2. 먹거리 주문")
    print("3. 좌석 조회 및 선택")
    print("4. 내 정보 및 장바구니 확인")
    print("5. 연령 확인 및 최종 결제")
    print("6. 사용 종료 및 좌석 반납")
```

```
print("7. [관리자 모드]")
print("="*30)

choice = int(input("원하시는 메뉴 번호를 입력하세요: "))

# 1. PC 시간 충전
if choice == 1:
    money = int(input("충전할 금액을 입력하세요 (원): "))
    add_time = money / 1000  # 1000원당 1시간

    # 10,000원 이상 충전 시 보너스 2시간 지급
    if money >= 10000:
        print("🎉 10,000원 이상 결제! 보너스 2시간이 추가됩니다!")
        add_time = add_time + 2

    total_time = total_time + add_time
    total_payment = total_payment + money
    print(f"> {add_time}시간이 충전되었습니다. (총 남은 시간: {total_time}시간)")

# 2. 먹거리 주문
elif choice == 2:
    print("\n[ 먹거리 메뉴판 ]")
    # for문을 이용해 메뉴판 출력
    for j in range(4):
        print(f"{j+1}번: {food_menu[j]} ({food_price[j]}원)")

    food_choice = int(input("주문할 음식의 번호를 선택하세요 (취소는 0): "))

    if food_choice >= 1 and food_choice <= 4:
        # 리스트는 0번부터 시작하므로 -1을 해줍니다.
        selected_food = food_menu[food_choice - 1]
        selected_price = food_price[food_choice - 1]

        cart.append(selected_food) # 장바구니(리스트)에 추가
        total_payment = total_payment + selected_price
        print(f"> {selected_food}이(가) 장바구니에 담겼습니다!")
    elif food_choice == 0:
        print("> 주문을 취소합니다.")
    else:
        print("> 잘못된 번호입니다.")

# 3. 좌석 조회 및 선택
elif choice == 3:
```

```python
    print("\n[ 현재 좌석 현황 ]")
    for j in range(10):
        print(f"{j+1}번 좌석: {seats[j]}")

    seat_choice = int(input("원하시는 좌석 번호를 선택하세요 (1~10): "))

    if seat_choice >= 1 and seat_choice <= 10:
        index = seat_choice - 1 # 리스트 인덱스 맞추기

        if seats[index] == "비어있음":
            seats[index] = "사용중"
            my_seat = seat_choice
            print(f"> {my_seat}번 좌석이 배정되었습니다. 즐거운 시간 보내세요!")
        else:
            print("> ❌ 이미 사용 중인 좌석입니다. 다른 좌석을 선택해 주세요.")
    else:
        print("> 잘못된 좌석 번호입니다.")

# 4. 내 정보 및 장바구니 확인
elif choice == 4:
    print("\n=== [ 내 정보 요약 ] ===")
    if my_seat == -1:
        print("좌석: 아직 선택하지 않았습니다.")
    else:
        print(f"좌석: {my_seat}번 좌석 사용 중")

    print(f"남은 시간: {total_time}시간")

    print("장바구니 목록: ", end="")
    if len(cart) == 0:
        print("비어있음")
    else:
        # for문을 이용해 장바구니 내용물 나열
        for item in cart:
            print(f"[{item}] ", end="")
        print() # 줄바꿈

    print(f"현재 총 결제 예정 금액: {total_payment}원")
    print("========================")

# 5. 연령 확인 및 최종 결제
elif choice == 5:
    print(f"\n결제하실 금액은 총 {total_payment}원 입니다.")
```

```
        time_check = input("현재 시간이 밤 10시(22시) 이후입니까? (네/아니오): ")

        if time_check == "네":
            age = int(input("신분증 확인을 위해 나이를 숫자로 입력해주세요: "))
            if age < 19:
                print("> 🚨 경고: 미성년자는 야간 출입이 불가합니다. 결제가 차단됩니다.")
                # 미성년자 차단 시 결제를 진행하지 않고 넘김
                continue

        # 미성년자 야간이 아닌 경우 정상 결제 진행
        print("> 결제가 완료되었습니다. 감사합니다!")
        daily_revenue = daily_revenue + total_payment # 사장님 매출에 누적
        total_payment = 0 # 결제했으니 내가 낼 돈은 0원으로 초기화
        cart = [] # 장바구니 비우기

    # 6. 사용 종료 및 좌석 반납
    elif choice == 6:
        print("\n이용을 종료합니다.")
        if my_seat != -1:
            seats[my_seat - 1] = "비어있음" # 좌석 반납
            print(f"> {my_seat}번 좌석이 정상적으로 반납되었습니다.")

        print(f"> 고객님의 남은 시간은 {total_time}시간입니다. 다음에 또 오세요!")
        break # for 반복문을 즉시 깨고 프로그램을 완전히 종료합니다!

    # 7. 관리자 모드
    elif choice == 7:
        pw_input = input("관리자 비밀번호를 입력하세요: ")
        if pw_input == admin_pw:
            print("\n=== [ 사장님 전용 메뉴 ] ===")
            print(f"오늘의 총 누적 매출: {daily_revenue}원")
            print("============================")
        else:
            print("> ❌ 비밀번호가 틀렸습니다.")

    else:
        print("> 메뉴에 없는 번호입니다. 다시 선택해주세요.")
```

이 코드를 복사해서 파이썬 환경(IDLE이나 구글 코랩 등)에서 바로 실행해 보시면 직접 번호를 누르면서 테스트해 보실 수 있습니다. 제약 조건을 완벽하게 지키면서도 요구사항을 모두 담아낸 아주 멋진 1학기 최종 목표 코드가 완성되었습니다. 과제 진행하시면서 막히는 부분이

생기면 언제든지 다시 찾아와 주세요!

사용자별로 각자의 시간을 따로 저장하고 관리하는 기능이라니! 이건 정말 실제 서비스되는 PC방 회원가입 시스템과 똑같은 수준의 아주 훌륭한 아이디어입니다.

아직 딕셔너리(사전)라는 자료형을 배우지 않으셨다면, 허락된 도구인 **리스트 2개를 나란히 사용하는 방법(병렬 리스트)**으로 완벽하게 구현할 수 있습니다.

원리는 아주 간단합니다. **'회원 이름 리스트'**와 **'남은 시간 리스트'**를 따로 만들어서, 같은 순서(인덱스)에 데이터를 저장하는 것입니다.

💡 사용자별 시간 저장 원리 (병렬 리스트)

만약 '철수'가 처음 가입하고, 그다음에 '영희'가 가입했다면 리스트는 이렇게 됩니다.

- user_names = ["철수", "영희"]
- user_times = [3.0, 5.0]
  철수는 0번째 인덱스에 있으니, 철수의 시간도 0번째 인덱스인 3.0시간이 되는 방식이죠!

💻 1번 메뉴 코드 적용 (사용자별 시간 저장)

이 원리를 적용해서 1번 메뉴 코드를 수정하면 이렇게 됩니다. 리스트의 .append()(추가)와 .index()(위치 찾기) 메소드를 아주 기가 막히게 활용할 수 있습니다.

Python

```python
# --- [초기 설정: 맨 위에 리스트 만들기] ---
user_names = []  # 사용자(회원) 이름을 저장할 빈 리스트
user_times = []  # 사용자별 잔여 시간을 저장할 빈 리스트

# ... (메인 메뉴 반복문 시작) ...

  # 1. PC 시간 충전
  if choice == 1:
    name = input("회원 이름을 입력하세요: ")
    money = int(input("충전할 금액을 입력하세요 (원): "))

    add_time = money / 1000  # 1000원당 1시간
    if money >= 10000:
      print("🎉 보너스 2시간 추가!")
      add_time = add_time + 2

    # [핵심] 신규 회원인지 기존 회원인지 리스트를 확인합니다!
    if name in user_names:
```

```
        # 1. 기존 회원일 경우: 이름 리스트에서 몇 번째(인덱스)에 있는지 찾습니다.
        user_index = user_names.index(name)

        # 2. 시간 리스트의 똑같은 위치에서 기존 시간을 꺼내 더해줍니다.
        previous_time = user_times[user_index]
        user_times[user_index] = previous_time + add_time

        print(f"> {name}님 다시 오셨군요! (기존 {previous_time}시간 + 충전 {add_time}시간)")
        print(f"> 총 잔여 시간: {user_times[user_index]}시간")

    else:
        # 신규 회원일 경우: 두 리스트에 각각 이름과 시간을 새롭게 추가(append)합니다.
        user_names.append(name)
        user_times.append(add_time)

        print(f"> {name}님 환영합니다! 신규 회원으로 등록되었습니다.")
        print(f"> 총 잔여 시간: {add_time}시간")
```

이렇게 코드를 짜면 교수님께서 "리스트의 인덱스 개념과 메소드를 완벽하게 이해하고 응용했네!" 하고 극찬하실 만한 포인트가 됩니다.

이 방식을 적용하면, 나중에 4번 '내 정보 확인' 메뉴에서도 내 이름을 검색해서 나만의 남은 시간을 정확하게 띄워줄 수 있겠죠? 이 병렬 리스트 방식을 4번 메뉴(정보 확인)에도 어떻게 적용하면 좋을지 코드로 한번 연결해 볼까요?

아! 정말 훌륭한 테스트를 해보셨네요! 여러 명의 사용자가 번갈아 가며 키오스크를 사용하는 실제 상황을 가정하고 치명적인 논리 오류(버그)를 정확히 찾아내셨습니다. 코딩 실력이 쑥쑥 늘고 있다는 증거입니다!

이 문제가 발생하는 원인과 이를 해결하는 두 가지 방법을 명확하게 짚어드릴게요.

🔍 원인 분석: **my_seat** 변수의 '덮어쓰기' 현상

현재 코드 위쪽을 보면 my_seat = -1 이라는 변수가 딱 하나만 존재합니다.

1. A가 1번을 고르면 my_seat는 0이 됩니다.
2. B가 6번을 고르면 my_seat는 5로 덮어씌워집니다.
3. C가 10번을 고르면 my_seat는 최종적으로 9로 덮어씌워집니다.

이 상태에서 A가 6번 메뉴(종료)를 누르면, 컴퓨터는 "A가 누구지? 난 모르겠고, 지금 내 기억 속에 남은 my_seat는 9(10번 자리)니까 10번 자리를 비워야겠다!"라고 판단해 버리는 것입니다.

이 문제를 해결하려면 다음 두 가지 방법 중 하나를 선택해야 합니다.

🛠️ 해결책 **1:** 가장 쉽고 빠른 방법 **(**반납할 좌석 직접 묻기**)**

현재 코드를 가장 적게 고치면서 문제를 완벽히 해결하는 방법입니다. 6번 메뉴에서 컴퓨터가 my_seat 변수에 의존하지 않게 하고, **"**몇 번 좌석을 반납하시겠습니까**?"**라고 사용자에게 한 번 더 물어보게 만드는 것입니다.

6번 메뉴의 코드를 아래와 같이 수정해 보세요.

Python

```python
# 6. 사용종료 및 좌석 반납
elif num_choice == 6 :
    print("\n이용을 종료합니다.")
    name = input("종료하실 회원 이름을 입력해주세요 : ")

    if name in user_names :
        user_index = user_names.index(name)
        use_time = int(input("사용한 시간을 입력해주세요 : "))

        current_time = user_times[user_index]
        updated_time = current_time - use_time
        user_times[user_index] = updated_time
```

```
        # [수정된 부분] 반납할 좌석 번호를 직접 입력받아 처리합니다!
        return_seat = int(input("반납하실 좌석 번호(1~10)를 입력해주세요 : "))
        return_idx = return_seat - 1

        if seats[return_idx] == "사용중":
            seats[return_idx] = "비어있음"
            print(f"=> {return_seat}번 좌석이 정상적으로 반납되었습니다.")
        else:
            print(f"=> {return_seat}번 좌석은 이미 비어있습니다.")

    else :
        print(f"=> 등록되지 않은 이름입니다. 이름을 확인해 주세요.")
        continue # 이름이 없으면 여기서 바로 위로 돌려보냅니다.

    # (기존에 있던 if my_seat != -1: 부분은 과감히 지워주세요!)

    print(f"=> {name}님, 남은 시간은 {updated_time}시간 입니다. 안녕히 가세요!")
    continue
```

이렇게 하면 A가 와서 "1번 자리 반납할게"라고 직접 입력하므로 엉뚱한 자리가 비워지는 일이 사라집니다.

🚀 해결책 2: A+를 위한 완벽한 방법 (사용자별 좌석 리스트 만들기)

만약 "손님이 자기 자리 번호를 까먹을 수도 있잖아요! 컴퓨터가 알아서 A가 1번 자리라는 걸 기억하게 하고 싶어요"라고 생각하신다면, 앞서 시간을 저장했던 방식(user_times)을 그대로 응용하면 됩니다.

1. 맨 위에 user_seats = [] 라는 리스트를 하나 더 만듭니다.
2. 3번 메뉴(좌석 선택)에서 이름을 먼저 입력받은 뒤, 그 사람의 인덱스 위치에 좌석 번호를 저장해 둡니다.
3. 6번 메뉴에서 이름을 입력하면, user_seats 리스트에서 그 사람의 좌석을 찾아 자동으로 비워줍니다.

💡 어떻게 적용하실 건가요?

현재 작성하신 코드의 전체적인 흐름(장바구니나 결제 금액도 변수가 1개뿐인 점)을 보았을 때, 키오스크의 특성상 **[해결책 1]**을 적용하는 것이 문맥상 가장 자연스럽고 오류가 없습니다.

해결책 1로 먼저 코드를 수정해서 테스트해 보시겠어요? 만약 해결책 2번 방식(리스트 추가)으로 아예 구조를 업그레이드하고 싶으시다면, 3번과 4번 메뉴까지 싹 다듬어 드릴 수 있습니다!